Technical Report: Operation DBU: Reloaded

1. Problem Statement

Create a fully playable 2D platformer set on the campus of Dallas Baptist University. The game must:

- Include **five distinct levels** with increasing difficulty.
- Feature **three enemy types** (Slow Student, Fast Student, Flying Drone).
- Allow the player to collect **Access Chips** (required to unlock the exit) and **Patriot Tokens** (bonus points).
- Implement a **lives system** (3 lives, invincibility frames after hit).
- Support **persistent scoring** and a **leaderboard**.
- Use **procedurally generated sprites** (no external image assets) – but optionally accept user-provided photo backgrounds.
- Run smoothly with **camera scrolling**, **collision detection**, and **audio**.
- Be robust against crashes and include a **polished user interface** (title screen, mission briefings, game over, victory).

The project was developed by a team of four students over several weeks, using iterative testing and feedback to refine gameplay and fix bugs.

2. Data Source and Preprocessing

Unlike a typical data-driven project, the “data” comprises:

- **Level layouts** – hand-crafted tile maps stored as Python lists in `game/levels.py`. Each tile is a 40×40 pixel block (grass, stone, shelf, metal). Platform placements, collectible coordinates, enemy patrol ranges, and exit locations are all written as plain Python data structures.

- **Background images** (optional) – players can drop campus photos into the `backgrounds/` folder. The game automatically:

  - Validates aspect ratio (allowed: 16:9, 16:10, 4:3 with ±6% tolerance).

  - Scales the image to fit the game window (960×490, HUD excluded).

  - Compresses the result to a `.cache.jpg` (quality 72) for fast loading on subsequent launches.

  - If the image is rejected (wrong ratio, corrupt file), the game falls back to a procedural background.

- **User feedback data** – manual playtesting by team members (Noah, Tim, Juan) produced a bug list. These qualitative reports (e.g., “ESC should return to main menu, not quit”, “victory crashes”, “rename debug menu”) drove improvements in `game_engine.py`, `screens.py`, and `scoreboard.py`.

Preprocessing steps:

- Sprites are drawn at runtime using `pygame.draw` primitives – no image files needed.

- The background loader caches processed images so subsequent loads are instant.

- Level data is validated by unit tests (see `test_game.py`) that check chip counts, enemy counts, and tile generation.

3. Core Implementation

Technology stack:

- Python 3.9+

- Pygame 2.0+

- No other external libraries (Pillow is optional for better JPEG compression).

Architecture – modular, object-oriented design:

game/

├── game_engine.py # State machine & main loop

├── player.py # Agent Patriot (movement, collisions, lives)

├── enemies.py # WalkingEnemy base, SlowStudent, FastStudent, FlyingDrone

├── collectibles.py # AccessChip, PatriotToken, ExitPortal

├── levels.py # Level data + Tile sprite

├── sprites.py # Procedural art functions

├── hud.py # On-screen HUD (lives, score, chips)

├── screens.py # Title, briefing, level complete, game over, victory

├── background_loader.py # Photo background handling

├── audio_manager.py # SFX, intro stings, BGM (all optional)

└── scoreboard.py # Name entry + persistent JSON leaderboard

Game states (finite state machine in `game_engine.py`):

`STATE_TITLE` → `STATE_TUTORIAL` / `STATE_SCOREBOARD` → `STATE_BRIEFING` → `STATE_PLAYING` → `STATE_LEVEL_DONE` (repeat) → `STATE_VICTORY` → `STATE_TITLE`.

Physics and collisions:

- Gravity = 0.55 pixels/frame², jump force = -13, horizontal speed = 4.

- Collision resolution: separate X and Y passes using tile rects.

- Platforms are `Tile` sprites; thin platforms (half-height) also supported.

Camera:

- Horizontal scrolling: `camera_x = max(0, min(target, level_width - SCREEN_WIDTH))`.
- Background parallax: optional photo backgrounds scroll at 25% of foreground speed.

Enemy AI:

- `SlowStudent` / `FastStudent` – patrol horizontally between defined left/right boundaries.
- `FlyingDrone` – patrol vertically between top/bottom boundaries.
- All enemies cause the player to lose one life on contact (with invincibility frames).

Collectibles and progression:

- Access Chips: required to unlock exit; 500 points each.
- Patriot Tokens: bonus points only; 100 points each.
- Exit portal: becomes active only after `chips_collected >= chips_needed`.

Audio (`audio_manager.py`):

- **Intro stings** (assets/sounds/intro/levelN.ogg) – played once during mission briefing.
- **BGM** (assets/music/levelN_bgm.ogg) – looped during gameplay. If missing, gameplay continues silently.
- **SFX** – hit_prof, hit_students, hit_drones, opening, level_complete, game_completion.
- All audio files are optional; the game never crashes if a file is absent.

Persistence:

- Scoreboard stored in `saves/scoreboard.json` (max 10 entries, sorted by score).
- After completing level 5, a name-entry screen appears. The name and final score are appended to the leaderboard.
- Scores are displayed on the scoreboard screen (accessible from the title menu with `S`).

4. Results / Evaluation

Functionality:

- All five levels are completable without crashes.
- Player can move, jump, collect items, defeat enemies (by avoiding them), and exit each level.
- Lives system works: losing all three lives triggers game over.
- Score accumulates across levels; chips and tokens grant points.
- The level-select menu (triggered by `G`) allows jumping to any level, which aids testing.
- Mute (`M`) toggles all audio.

Testing:

- **Unit / smoke tests** (`tests/test_game.py`) run headless and verify:
  - Sprite sizes are correct.
  - Player hit reduces lives and enables invincibility.
  - Enemies stay within patrol ranges.
  - Level data integrity (chip count, tile generation).

  - Score constants are positive.

  - All tests pass.

- **Manual playtesting** (three team members across Windows) uncovered critical issues that were fixed:

  - **Issue #2**: ESC quits the game → changed to return to main menu.

  - **Issue #6**: Falling off screen respawned at incorrect location → fixed respawn logic.

  - **Issue #8**: Victory after level 5 crashed → added name-entry and scoreboard flow.

  - **Issue #9**: “Debug” menu misleading → renamed to “Select a Level” and improved UI.

  - **Background tiling** and missing shelf graphics were corrected in `sprites.py`.

Performance:

- Solid 60 FPS on any modern machine (tested on Windows 10/11, Ubuntu 22.04).

- Memory usage stable; no leaks observed.

User experience:

- On-screen HUD shows lives (hearts), score, chips collected, level name, and mute indicator.

- Tutorial screen explains controls, enemies, and items.

- Mission briefings before each level set the narrative.

- Victory screen displays final score and offers return to main menu.

5. Challenges and Lessons Learned

| Challenge | Solution | Lesson |

|-----------|----------|--------|

| **Audio loop interference** between levels and menus | Centralised `audio_manager` with separate categories (intro, BGM, SFX) and fade-out on state transitions. | Keep audio logic in one place; use `fadeout` to avoid abrupt cuts. |

| **User-provided background images** cause crashes if wrong ratio | Validate aspect ratio at load; scale with letterbox; cache as JPEG; fallback to procedural backdrop. | Graceful degradation is better than crashing. |

| **Player falling through thin platforms** | Separate X and Y collision checks; reset ground flag after each collision. | Test edge physics with multiple platform heights. |

| **Victory crash after level 5** | Missing name-entry screen. Added `_trigger_victory()` that calls `NameEntry.run()` and updates scoreboard before showing victory. | Every state transition must be explicitly coded – never assume. |

| **“Debug” label confuses users** | Renamed to “SELECT A LEVEL” and added background status (photo loaded / rejected / missing). | User-facing text should reflect actual function. |

| **Team coordination across different schedules** | Used GitHub, daily chat updates, and shared bug-tracking via instructor notes. Regular small commits and weekend testing sessions. | Frequent integration prevents merge hell. Keep a shared bug list. |

Overall success: Operation DBU: Reloaded delivers a complete, entertaining platformer that meets all project requirements. The combination of procedural art, optional photo backgrounds, and robust state management makes it easy to extend (new levels or enemies can be added by editing `levels.py` and `enemies.py`). The team learned valuable lessons in game loop design, collision detection, and user-driven debugging.

Repository: [GitHub – Garkair/DBU_GAME_GROUPC](https://github.com/Garkair/DBU_GAME_GROUPC)

Acknowledgments: Special thanks to Noah, Tim, and Juan for extensive playtesting and bug reports.